N8N Information Agent

> 基于 N8N、DeepSeek、MySQL 与飞书 Open API 构建的持续信息聚合与任务执行 Workflow。

本项目起源于一个朴素的追问：**大模型除了在网页里做一次性的问答，能不能进入一个长期运行、自定义、操作固定的自动化流程，持续完成"信息获取 → 语义处理 → 结构化输出 → 消息分发 / 安排日程"的闭环?**

带着这个问题，我基于 N8N 设计并部署了一套端到端的智能信息处理 Agent。它是一个按照预设节奏持续运转、将 LLM 作为能力节点嵌入确定性工作流的自动化系统。

当前公开仓库展示的是实际运行版本的脱敏内容，包括 Workflow 结构、Prompt、SQL Schema、JavaScript 消息处理代码及运行结果截图。

1. 系统概览

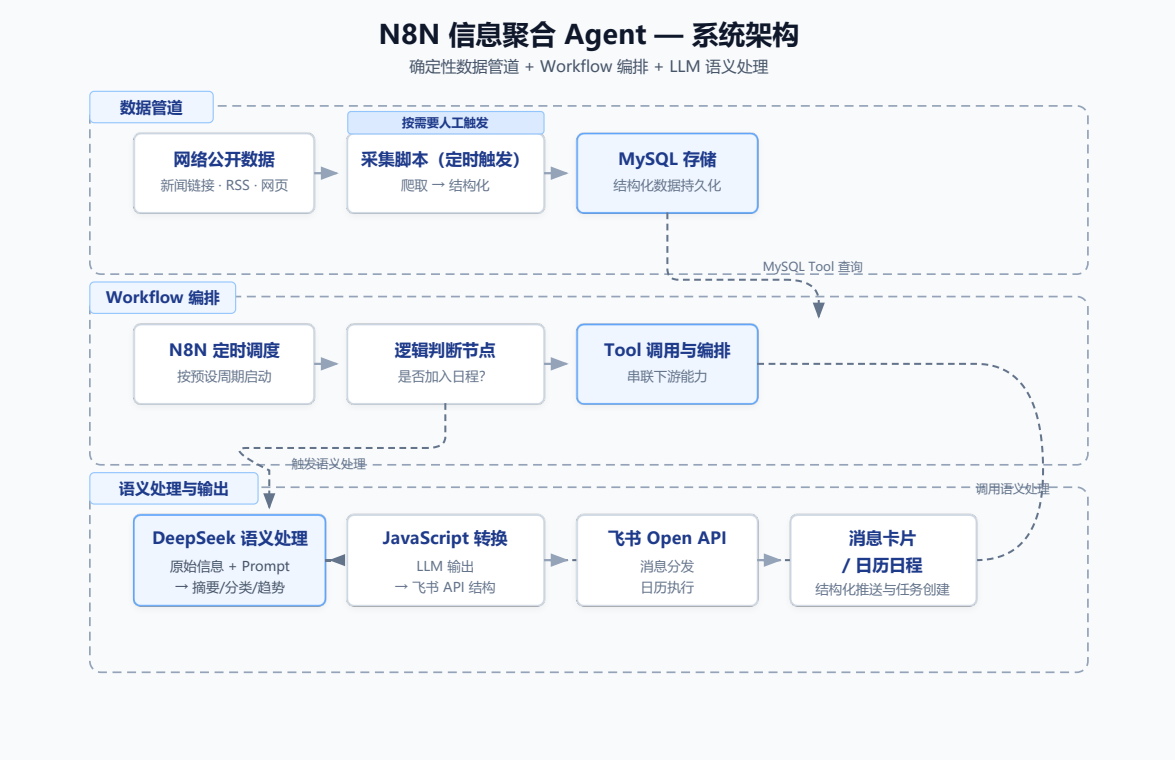
该 Workflow 按预设时间自动启动，完成以下链路：

1. **N8N Schedule Trigger** 定时触发任务
2. **MySQL 查询节点**读取近期新闻与活动候选数据
3. **DeepSeek Agent**分别承担新闻摘要及活动结构化信息提取
4. **JavaScript 节点**将模型输出转换为飞书 API 所需的消息结构
5. **HTTP Request** 调用飞书 Open API 推送结构化消息卡片
6. 在需要时进一步调用日历 API 创建日程任务

N8N 主工作流可按预设时间无人值守运行，完成已有数据的查询、语义处理、消息分发及日程创建；新闻采集脚本目前独立人工触发。调试时点击start.bat直接启动docker并浏览器打开节点画布。



2. 架构设计：模型只负责最擅长的语义理解与生成



系统采用 **“确定性数据管道 + LLM 语义处理 + Workflow 编排”** 的三层架构：

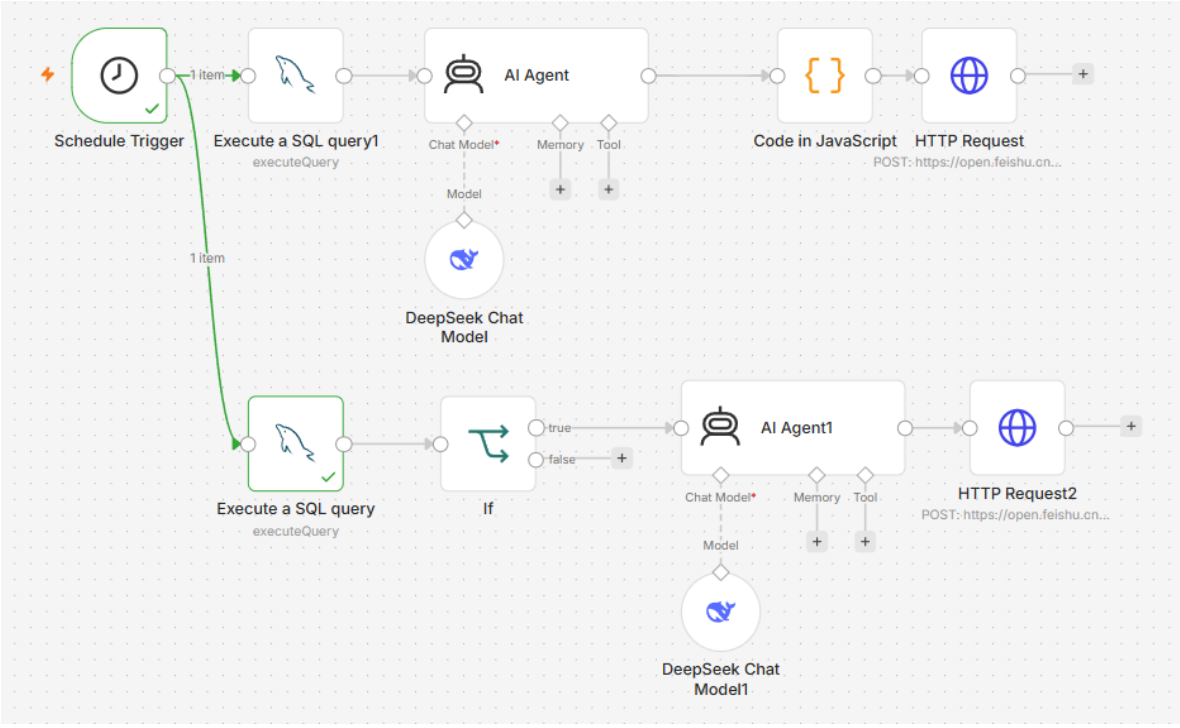
- **MySQL**：负责结构化信息的存储与查询，确保数据层稳定可控
- **DeepSeek**：承担摘要生成、关键词提取、主题分类与趋势总结等语义任务
- **N8N**：负责定时调度、节点编排和 Tool 调用，串联整个流程
- **JavaScript**：作为模型输出与外部 API 之间的确定性转换层，解耦 LLM 与飞书
- **飞书 Open API**：负责最终消息分发及日历任务执行

这套架构的核心意图是：**让大模型成为一个能力节点，绝不会作为流程的全部。** 调度、查询、格式转换、消息发送均由确定性程序控制，模型只负责最擅长的语义理解与生成。

3. Workflow 核心链路

当前 Workflow 的主要节点链路如下：

```
                Schedule Trigger
                /              \
               ↓                ↓
MySQL 新闻查询          MySQL 活动候选查询
               ↓                ↓
新闻摘要 Agent          Regex 条件筛选
               ↓                ↓
JavaScript Formatter    新闻详情页请求
               ↓                ↓
飞书消息卡片            活动信息提取 Agent
                                ↓
                          飞书日历事件
```



公开的脱敏 Workflow 文件位于：
information_agent.sanitized.json

> 公开版本已删除所有敏感配置，包括 API Key、Credential、Webhook、数据库密码等，仅保留节点结构与逻辑关系。

4. Prompt 设计：结构化输出

Agent 的 System Prompt 核心约束包括如何处理已知信息和结构化稳定输出

公开 Prompt 文件：
promptsnews_summary_prompt.md

promptsactivity_extraction_prompt.md

Prompt 本身只负责"模型该输出什么"，不涉及调度逻辑、数据库查询或消息发送。

5. 数据采集与持久化

所有信息数据使用 MySQL 进行结构化存储。Agent 通过 MySQL Tool 获取所需字段后再进行语义处理，而非完全依赖模型自身知识生成事实内容——这极大降低了模型幻觉对输出质量的影响。

其中数据采集公开采集脚本：

spider.py

当前版本通过 Requests + BeautifulSoup 抓取公开新闻页面，
解析标题、发布日期和原文链接，并按标题去重后写入 MySQL。

采集任务目前由人工触发，尚未纳入自动重试与异常调度。

公开仓库提供数据库 Schema 与示例查询，不包含真实运行数据：

- sql_schema.sql
- sql_example_queries.sql

6. 消息格式化：确定性程序与 LLM 的解耦层

模型生成的结构化输出，经由 JavaScript 节点进一步处理，转换成飞书 API 所要求的精确消息格式。这一层的存在意义是：

- **将大模型的自然语言输出与外部应用 API 解耦**
- **让消息结构由确定性程序控制，而非依赖模型的输出稳定性**
- **便于后续调整消息样式时，只需修改 JS 逻辑，无需改动 Prompt**

公开代码：
format_feishu_message.js

7. 运行结果

实际运行时，系统能够按照设定周期自动生成结构化信息卡片，并将结果推送至飞书。



8. 可靠性问题与已知局限

当前版本已经能够完成定时查询、模型处理与自动推送，但持续运行过程中暴露出可靠性不足。

早期运行中曾出现过一个典型情况：**上游数据库没有返回有效新闻数据，但 Workflow 仍继续进入 Agent 生成与消息推送阶段**，导致推送了无实际内容的消息。

这类问题说明，一个持续运行的 Agent Workflow，除了模型能力之外，还需要在工程层面进一步加强：

- 空数据检测与早期终止
- 重试与失败分支设计
- 防重复推送机制
- 完善的 Execution Log

9. 仓库结构

```
.
├── pics
|   ├── architecture.svg
|   ├── feishu_result.png
|   ├── n8n_panel.png
|   └── n8n_workflow.svg
├── prompts
|   ├── promptsactivity_extraction_prompt.md
|   └── promptsnews_summary_prompt.md
├── scripts
|   ├── format_feishu_message.js
|   └── spider.py
├── sql
|   ├── sql_example_queries.sql
|   └── sql_schema.sql
├── workflow
|   └── information_agent.sanitized.json
├── .env.example
├── .gitignore
├── README.md
└── start.bat
```

10. 后续迭代方向

当前仓库记录的是信息聚合 Agent 的第一阶段实现。后续计划包括：

1. **完善可靠性工程**：补充空数据检测、执行日志与异常分支处理
2. **增加数据分析能力**：对归档数据进行长期趋势分析与统计
3. **切换到消费级硬件行情追踪**：将工作流扩展至显卡、内存、SSD 等价格数据的持续监测
4. **引入定量分析与归因**：结合价格趋势、异常波动检测与同期行业新闻，完成基于证据的价格变化归因

> Roadmap 中列出的功能尚未纳入当前项目成果，属于后续迭代方向。